

Transaction Handling in Distributed Systems

2-Phase Commit · 3-Phase Commit · Saga Pattern — Study Notes

1. What is a Transaction?

A transaction is a set of database operations that must be performed together as a single unit of work. Either all of the operations succeed, or none of them do.

The ACID Properties

Every transaction should follow four properties, known together as ACID:

- **Atomicity** — All operations in the transaction either succeed together or fail together. There is no “half-done” state.
- **Consistency** — The database must be in a valid (consistent) state before the transaction starts and after it finishes. The transaction should never leave the data in a broken state.
- **Isolation** — When multiple transactions (T1, T2, T3) run at the same time, the final result should look as if they ran one after another (serially). Each transaction behaves as if it is running alone.
- **Durability** — Once a transaction is successfully committed, the data is permanently saved. Even if the database crashes right after, the committed data is not lost.

Note that transactions do not actually run one after another — they run concurrently (in parallel) for performance. Isolation only guarantees that the final result is the same as if they had run sequentially. This is called “serializability”.

The Problem in Distributed Systems

In a single database, ACID is handled by the database engine itself. But in a distributed system (for example, microservices where the Order service and Inventory service each have their own database), a single business transaction spans multiple databases. Now the question is: how do we make sure all of them commit together or roll back together?

There are three common approaches, which we cover below:

1. Two-Phase Commit (2PC)
2. Three-Phase Commit (3PC)
3. Saga Pattern

2. Two-Phase Commit (2PC)

As the name says, this protocol works in two phases:

- **Phase 1 — Voting / Prepare phase:** everyone is asked if they are ready to commit.

- **Phase 2 — Decision / Commit phase:** based on the votes, a final decision (commit or rollback) is sent to everyone.

Who is Involved?

- **Application** — initiates the transaction (“here is my set of operations, please do them”).
- **Transaction Coordinator** — the central component that manages the whole transaction and makes the final decision.
- **Participant 1 (Order microservice)** — owns the order database.
- **Participant 2 (Inventory microservice)** — owns the inventory database.

How it Works (Happy Path)

1. The application sends the set of operations to the transaction coordinator.
2. The coordinator starts the transaction and sends each operation to the right participant (for example, an update query to the Order service and an update query to the Inventory service). Participants execute the work but do not commit yet.
3. **Phase 1 begins:** the coordinator asks every participant, “Are you prepared to commit?” (PREPARE message). Each participant replies YES (OK) or NO.
4. **Phase 2 begins:** the coordinator makes the decision. If every participant said YES, it sends COMMIT to all of them. If even one participant said NO, it sends ROLLBACK to all of them.
5. Participants apply the decision and reply DONE. The coordinator closes the transaction and returns the result to the application.

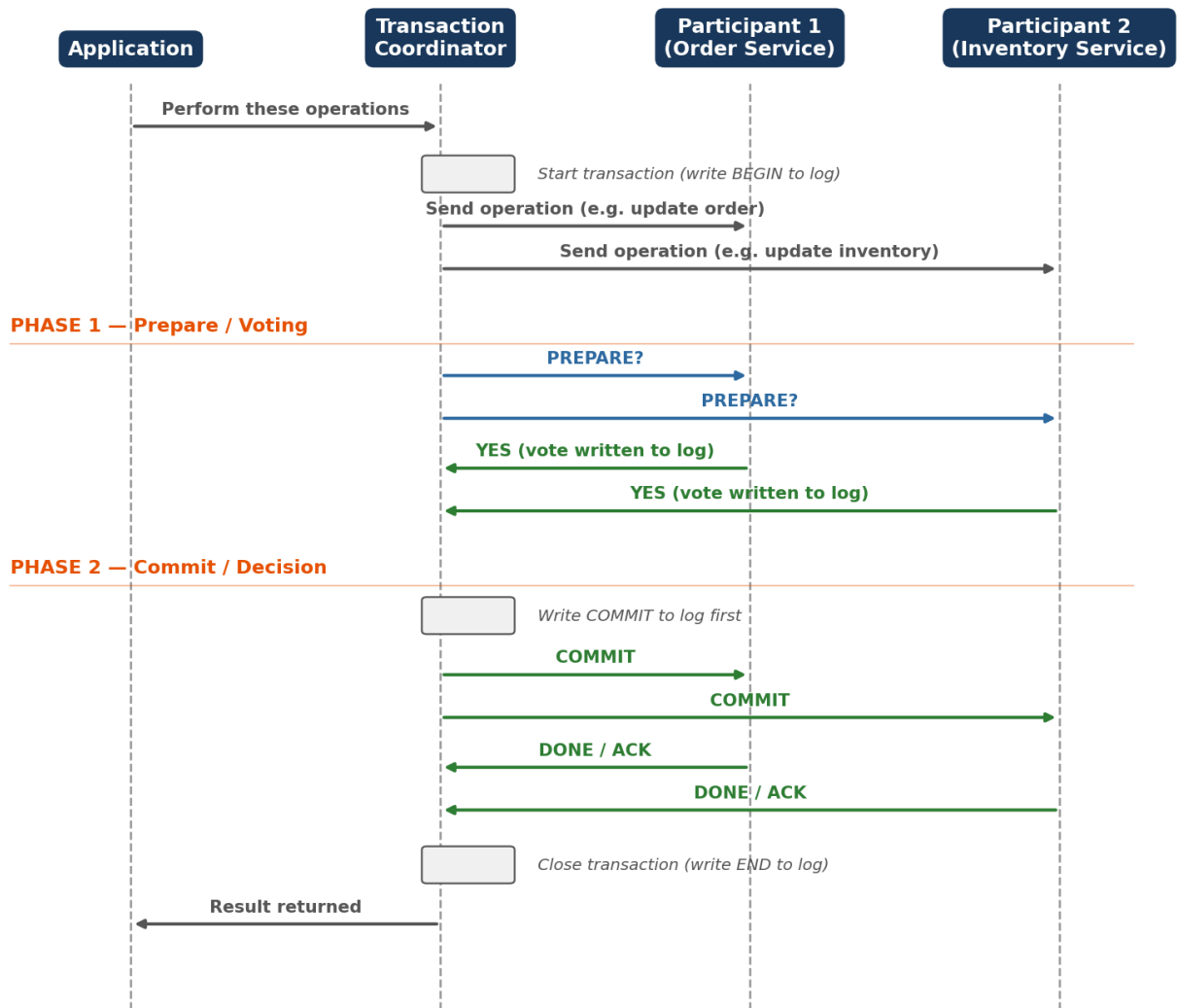


Figure 1 — Two-Phase Commit: successful (happy path) flow

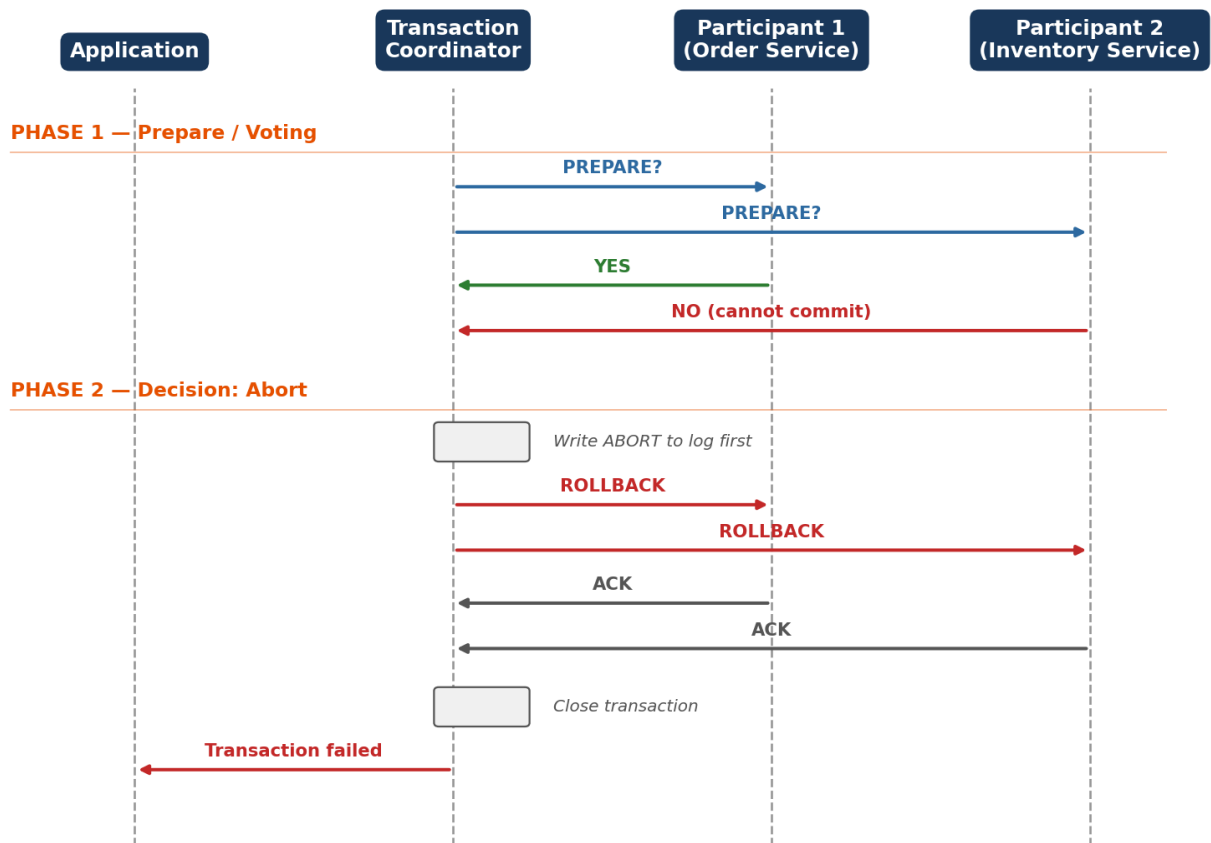


Figure 2 — Two-Phase Commit: one participant votes NO, so everyone rolls back

3. The Secret Behind Failure Recovery: Write-Ahead Logs

Before going into failure scenarios, we need one key idea. Both the coordinator and the participants maintain a log file (also called a write-ahead log or transaction log) on permanent storage such as disk, so it survives a crash:

- **The coordinator** writes every message to its own log file **before** sending it. For example, before sending PREPARE it logs “prepare sent”, and before sending COMMIT it first writes “commit” in the log, then sends the message.
- **Each participant** also maintains its own log file. It records what it received and what it voted (for example, “received prepare, voted YES”) before replying.

This rule — “**write to the log first, send the message after**” — is what allows both sides to recover after a crash. After restarting, a node simply reads its log to know exactly where the transaction was left.

4. Failure Scenarios (Common Interview Questions)

Q1. What happens if the PREPARE message is lost (or the coordinator fails before sending it)?

- The participants have already executed their operations and logged them, and they are waiting for the PREPARE message.
- Since PREPARE never arrives, each participant waits for a timeout period and then simply aborts (rolls back) the transaction on its own. This is safe, because nothing was committed yet.
- If the PREPARE message arrives late (after the participant has already aborted), the participant just replies NO, and the coordinator will roll back the whole transaction.

Q2. What happens if a participant's OK (YES vote) message is lost?

- Suppose Participant 1 sends its YES successfully, but Participant 2's YES is lost (or Participant 2 crashed before sending it).
- The coordinator waits for a timeout. Since it did not receive all votes, it decides to abort and sends ROLLBACK to everyone.
- When Participant 2 comes back up, it checks its log and sees an unfinished (uncommitted) transaction. It asks the coordinator, “What happened to this transaction?” The coordinator checks its own log and replies “Aborted”, so Participant 2 also rolls back.

Q3. What happens if the COMMIT message is lost?

A COMMIT message can be lost either because the coordinator went down after sending it partially, or because the network dropped it. The recovery idea is the same in both cases:

- Suppose there are three participants. The coordinator writes COMMIT into its log, sends COMMIT to P1 and P2, but P3 is down (or the message to P3 is lost).
- P1 and P2 commit, but P3 is now in a blocking state — it voted YES, so it is not allowed to abort on its own, and it has not received the decision, so it cannot commit either. It must wait.
- When P3 comes back up (or the coordinator recovers), P3 asks the coordinator for the decision. The coordinator reads COMMIT from its log and tells P3 to commit. Now everyone is consistent.

The Main Problem with 2PC

A participant that has voted YES **cannot make any decision on its own** — it fully depends on the coordinator. If the coordinator goes down after the participants voted YES but before the decision arrives, all participants are stuck (blocked), holding their locks, until the coordinator comes back. This is why 2PC is called a **blocking protocol**.

Interview tip: If asked “why is 2PC not used much in microservices?” — the answers are: it is blocking, the coordinator is a single point of failure, it holds locks for a long time (bad for performance), and it does not scale well over unreliable networks.

5. Three-Phase Commit (3PC)

3PC tries to fix the blocking problem of 2PC by adding one extra phase in the middle. So instead of two phases, we now have three:

1. **Phase 1 — CanCommit (Voting):** same as in 2PC. The coordinator asks every participant, “can you commit?” (PREPARE / CAN-COMMIT message), and each participant votes YES (OK) or NO.
2. **Phase 2 — PreCommit:** if all votes are YES, the coordinator sends a PRE-COMMIT message. This tells every participant, “everyone voted yes; get ready, the commit is coming.” Each participant acknowledges it (ACK).
3. **Phase 3 — DoCommit:** the coordinator sends the final COMMIT message, participants commit their changes and reply DONE. The coordinator then closes the transaction and returns the result to the application.

If any participant votes NO in Phase 1, or fails to acknowledge in Phase 2, the coordinator sends an ABORT to everyone — just like the rollback path in 2PC.

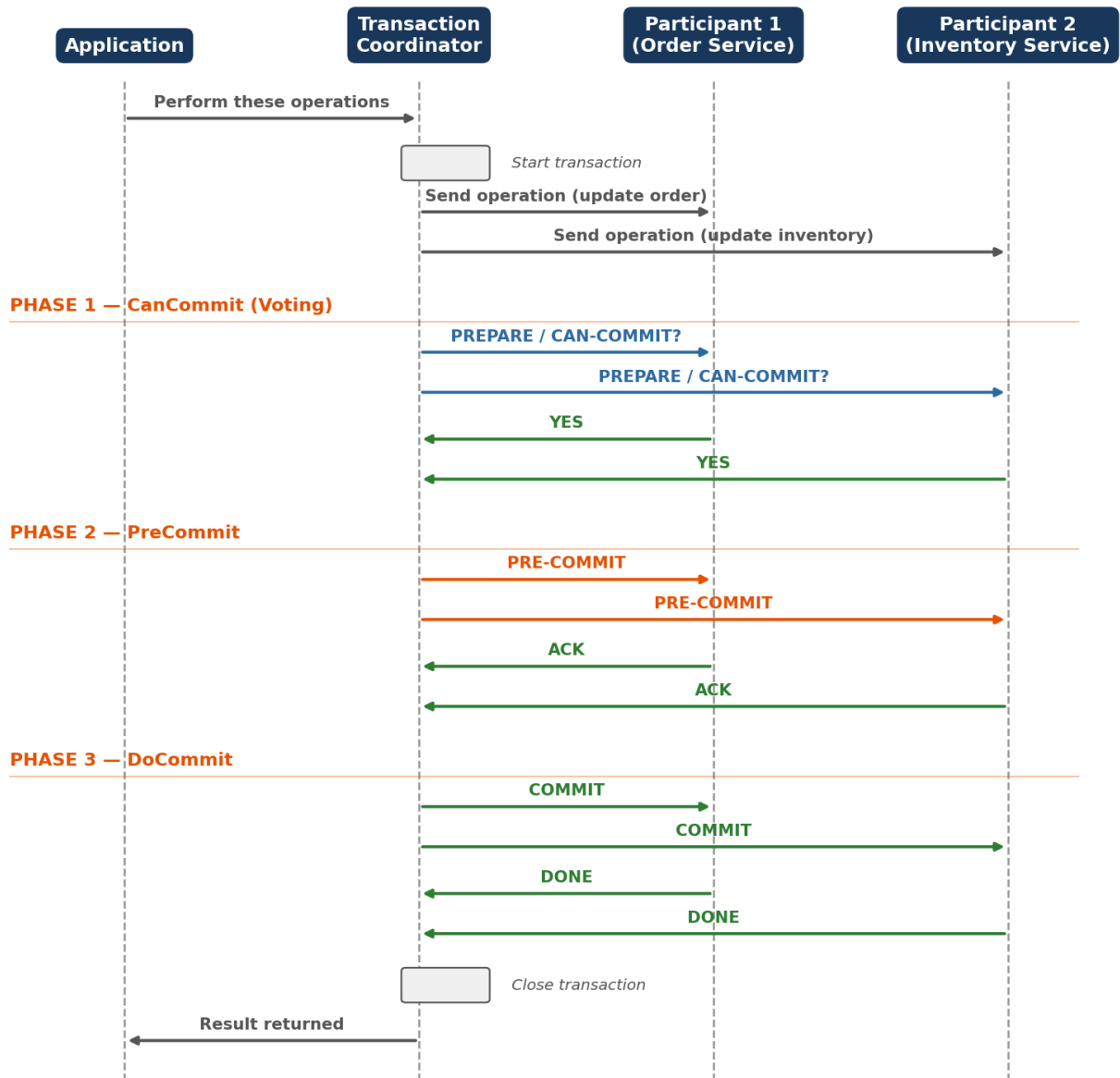


Figure 3 — Three-Phase Commit: CanCommit → PreCommit → DoCommit

Why Does the Extra Phase Help?

The PRE-COMMIT message carries an important piece of information: any participant that receives it now **knows that every other participant voted YES**. This changes what a participant can do when the coordinator crashes:

- In **2PC**, a participant that voted YES knows nothing about the other votes, so it must block and wait for the coordinator.
- In **3PC**, if a participant has received PRE-COMMIT and the coordinator then crashes, the participants can talk to each other and safely commit on their own after a timeout. If a participant has not received PRE-COMMIT, it can safely abort after a timeout, because it knows the commit decision was never made.

So 3PC is a **non-blocking protocol** (in the absence of network partitions) — participants are no longer completely stuck when the coordinator dies.

The Trade-offs of 3PC

- One extra round of messages for every transaction, so it is slower than 2PC.
- It can still behave incorrectly during a network partition (when the network splits into groups that cannot talk to each other, each group may take a different decision).
- Because of this extra cost and complexity, 3PC is rarely used in real production systems — but it is a very common interview topic.

6. Saga Pattern

Both 2PC and 3PC try to make a distributed transaction behave like one big atomic transaction, using locks and a coordinator. The Saga pattern takes a completely different approach — and it is the one most modern microservice systems actually use.

The Core Idea

A Saga breaks one big distributed transaction into a **chain of small local transactions**, one per service. Each service commits its own local transaction immediately — no global locks, no waiting for others.

If any step in the chain fails, the Saga runs **compensating transactions** — undo actions for the steps that already completed, executed in reverse order. For example: if the order was created and payment was charged, but reserving inventory fails, the Saga refunds the payment and cancels the order.

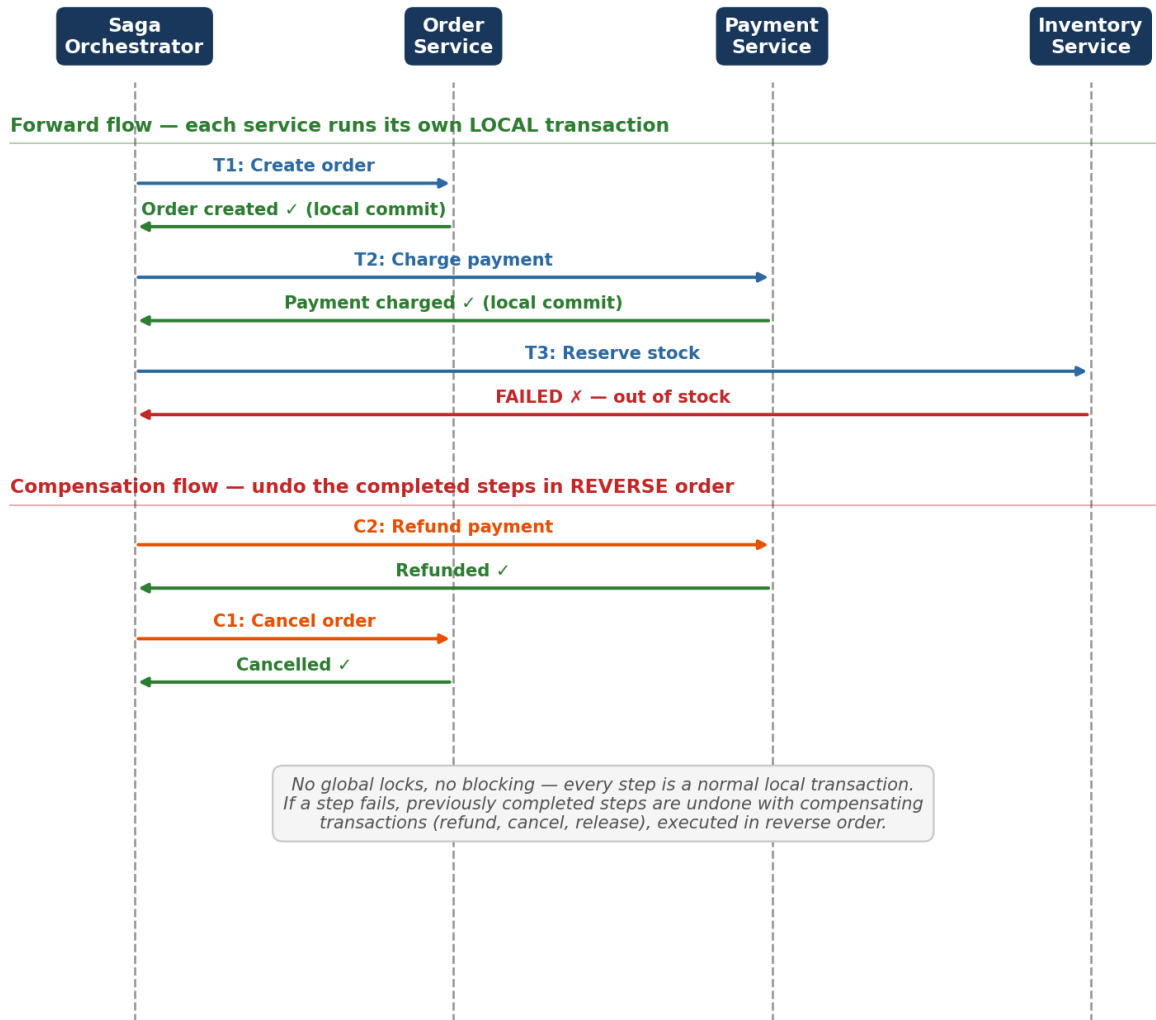


Figure 4 — Saga pattern: local transactions with compensation on failure

Two Ways to Implement a Saga

- **Choreography** — there is no central coordinator. Each service listens for events and reacts. For example, the Order service publishes an “OrderCreated” event, the Payment service hears it and charges the customer, then publishes “PaymentCompleted”, and so on. Simple for small flows, but hard to follow when many services are involved.
- **Orchestration** — a central Saga orchestrator tells each service what to do next and triggers the compensations when something fails (this is what Figure 4 shows). Easier to understand and debug, but the orchestrator is one more component to build and maintain.

The Trade-off: Eventual Consistency

Because each service commits its local transaction immediately, there are moments where the system is temporarily inconsistent — for example, the payment is charged but the stock is not yet reserved. The

system becomes consistent **eventually**, once the whole Saga finishes (or all compensations run). This is called **eventual consistency**, and it is the price we pay for avoiding locks and blocking.

Interview tip: A classic interview comparison: 2PC gives strong consistency but blocks and does not scale; Saga scales well and never blocks, but only guarantees eventual consistency and requires you to write a compensating (undo) action for every step.

7. Quick Revision Summary

- A transaction = a group of DB operations that must succeed or fail together (ACID).
- In distributed systems, one business transaction spans multiple databases, so we need a protocol like Two-Phase Commit to keep them in sync.
- 2PC = Prepare phase (voting) + Commit phase (decision), managed by a coordinator.
- Recovery works because everyone writes to a durable log file before sending any message.
- Lost PREPARE → participants time out and abort. Lost YES vote → coordinator times out and aborts. Lost COMMIT → participant blocks and asks the coordinator later.
- 2PC's weakness: it is blocking, and the coordinator is a single point of failure.
- 3PC = CanCommit + PreCommit + DoCommit. The extra PreCommit phase lets participants know that everyone voted YES, so they can decide among themselves if the coordinator crashes.
- 3PC reduces blocking but adds an extra message round and can still fail under network partitions, so it is rarely used in practice.
- Saga = a chain of local transactions; on failure, compensating (undo) transactions run in reverse order. Two styles: choreography (events) and orchestration (central orchestrator).
- Saga is non-blocking and scales well, but gives only eventual consistency — the practical choice for modern microservices.